Deliverable 45 — Free read-only measurement: the true binding ceiling on processing pace

*Measured 2026-10-20 live against **production** (Railway project `db04e5b1`, prod env `c1166ef6`), app + worker both running SHA `7ccf8f6` (= local HEAD = origin/main, worker deployment `82f9d80e`, SUCCESS). All observations are **read-only**: live `pg_stat_activity` / `SHOW` queries over the public DB proxy, and the worker's own `[DIAG]` telemetry pulled from Railway deployment logs. No writes, no env changes, no scoring batch, nothing that touches batch 1297.*

1. What was asked

Before committing to any replica / concurrency / pool change, find — empirically, for free — **which resource ceiling binds throughput first**: Postgres connections, the DeepSeek LLM account rate limit, search quota, or browser memory. The prior B6 inventory (`gate_b6_out/`, Oct 18) characterised the **DB** ceiling by arithmetic but left `connectionsPerJob` *derived* (not measured) and explicitly bracketed the **LLM account cap out of scope**. That cap was the suspected true first-binder. This measurement closes both gaps with live numbers.

2. Measurement window

The worker telemetry covers **23:34–00:11 UTC** (~37 min), the active second half of production batch **1297** (50 companies, framework 19). During 23:34–23:40 multiple companies were processing concurrently (Marriott, Stora Enso, …) — i.e. **real concurrency-4 load**, not idle — so the numbers below are load numbers, not a quiet baseline. By the tail (00:08+) the batch was down to its last company (China Three Gorges). Where a dimension could only be seen at low load, it is flagged.

3. Findings per resource (actual numbers)

3.1 Postgres connections — NOT binding, with large headroom

Metric	Measured	Source
`max_connections`	100	`SHOW` (live)
`superuser_reserved_connections`	3	`SHOW` (live)
Usable non-superuser	**97**	100 − 3
Worker main pool `pool_max` (per job child)	**4**	`[DIAG][db-op-sample]`
Main pool **in use** under load	**1–2** (mean 1.75)	`[DIAG][db-op-sample]`, 12 samples
Queued checkouts (`waiting`)	**0** (every sample)	`[DIAG][db-op-sample]`
`checkout_wait_ms`	**0** (max and mean)	`[DIAG][db-op-sample]`
Live backends on `companyiq_v3`	3–4	`pg_stat_activity`

The worker main pool runs at **1–2 of 4** connections per job child with **zero** queued checkouts and **zero** checkout wait — even during concurrent processing. The `pool_max=4` observed live is the **child-isolation partition** of `PG_POOL_MAX=20` (`floor((20 − parent_reserve) / WORKER_CONCURRENCY)` → `floor(16/4)=4`), which **empirically validates** B6's `connectionsPerJob ≈ 2` assumption. DB is nowhere near its 97-usable ceiling at current scale.

3.2 Per-replica LLM concurrency gate (`max=8`) — the only resource that showed back-pressure

`[DIAG][llm-permits] held=… waiting=… max=8`, 295 samples:

Metric	Measured
Gate size (per replica)	8
Held — mean / max	**0.60 / 8**
Samples at FULL (`held=8`)	**5 (2%)**
Samples with jobs WAITING (`waiting>0`)	**4 (1%)**
Max jobs waiting on a permit	**5**

This is the **only** gate observed to produce queueing. It saturates (`held=8`, up to 5 waiting) only **transiently** during bursts — 98% of the time it is below capacity, mean 0.6/8. Near-to-binding, but not binding at 1 replica.

3.3 DeepSeek LLM account rate limit (shared/global) — NOT hit; absolute ceiling UNMEASURED

Across the 37-min window at concurrency 4, there was exactly **one** real rate-limit event — `[AI] Fallback mistral-or: … status code 429` — on a *fallback* provider, not DeepSeek. DeepSeek primary **never rate-limited**. The other provider failures were **output truncation** (`finish_reason=length` / `max_tokens`), a content-length issue, **not** a rate ceiling.

Conclusion: at ~0.6 mean / 8 peak concurrent LLM calls on one replica, the DeepSeek account cap was **not reached**. We therefore know current demand is **below** the account ceiling, but its **absolute value is INCONCLUSIVE** — it cannot be extrapolated from a load that never touched it. Measuring it requires deliberately ramping concurrent calls until the 429 rate rises, which is **not** a read-only observation.

3.4 Browser pool / memory — NOT binding

Browser permit holders **max 1** (mean 1.0); policy-family slots **7/10** used with 173 non-policy slots free; **no OOM, SIGKILL, restart, or crash** in the window. Worker stable throughout.

4. The binding-ceiling ranking

At current production scale (1 worker replica, concurrency 4) **no resource binds hard** — every measured dimension has headroom. Ordered by proximity to binding:

Rank	Resource	Scope	State at concurrency 4	Binds first when…
1	Per-replica LLM gate (`max=8`)	per-replica	transient saturation (2% full, mean 0.6)	a single replica is pushed harder; scales out with replicas
2	**DeepSeek account rate limit**	**shared/global**	not hit (1 fallback-429)	**replica count × per-replica LLM demand** rises — the true first-binder at high fan-out, **absolute cap unmeasured**
3	DB connections (97 usable)	shared/global	1–2 of 4 per child, 0 waiting	~3 worker replicas at today's pool sizes (worst-case arithmetic)
4	Browser pool / memory	per-replica	idle (max 1 holder)	not a near-term constraint

5. Implication for scaling to thousands of companies

Horizontal scaling (your preference — more replicas, each individually stable) is bounded by the **shared** ceilings, #2 and #3, because per-replica gates (#1, #4) simply replicate with each worker:

- **DB (#3):** ~3 worker replicas at the current per-replica pool footprint before the 97-usable ceiling is approached (consistent with B6). This ceiling is **raisable** by lowering per-replica `PG_POOL_MAX` — exactly what the B6 `CAPACITY_POLICY` governs — so DB is a *tunable*, not a hard wall.
- **DeepSeek account (#2):** the genuine unknown. It was not stressed at 1 replica, but N replicas multiply concurrent DeepSeek demand against **one shared account**. Its absolute requests/min ceiling is the single most important number still missing, and it is the resource most likely to bind first at large replica counts.

So the honest state is: **DB is comfortable and tunable; the DeepSeek account cap is the next thing to measure, and it can only be measured under real multi-replica load** (the existing `[DIAG][llm-permits]` and 429 telemetry already emit the signal needed to read it).

6. Conclusive vs. inconclusive (so nothing is overstated)

- **Conclusive:** DB not binding (1–2/4, 0 waiting, 0 checkout wait); 97 usable; `connectionsPerJob ≈ 2` validated. LLM per-replica gate=8 saturates only transiently. DeepSeek account **not** hit during a full 50-company batch. No memory/OOM/restart; browser pool idle.

- **Inconclusive (honestly, not a failure):** the **absolute** DeepSeek account rate ceiling (only a lower bound is known); per-replica CPU/memory **quantiles** (no crash seen, but Railway metrics API not sampled this turn); peak DB/LLM at **sustained** full concurrency-4 across a whole batch (window captured bursts, not a long saturation plateau).

7. Recommended next step (one proposal)

Scale to 3 worker replicas and read the DeepSeek account ceiling off the telemetry that is already emitting — a bounded, in-production measurement of the one remaining unknown, with a pre-set rollback.

Rationale: DB comfortably supports 3 worker replicas (§5), so this step is DB-safe by the measured arithmetic. Tripling worker count triples concurrent DeepSeek demand against the shared account — the exact load that would reveal the account cap — while the existing `[DIAG][llm-permits]` (held/waiting) and `[AI] … 429` lines read it out live, for free. Pre-defined stop/rollback: if the 429 rate climbs materially or `llm-permits waiting>0` becomes sustained (not transient), drop back to the current topology; that is itself the measured account ceiling. This advances horizontal scaling and measures the unknown in a single bounded trial, with no code change — only a replica-count change, applied **only on your explicit go-ahead** (this turn’s work was the read-only measurement you authorised; the scale change is not yet made).